

# HygieneBench: A Reproducible Synthetic Benchmark for Cyber-Hygiene Anomaly Detection Across Identity, Endpoint, and Patch Telemetry

Anonymous Author(s)

## ABSTRACT

Cyber-hygiene anomaly detection—identifying stale privileged accounts, dormant account reactivations, endpoint coverage gaps, patch noncompliance clusters, and telemetry missingness—is operationally important but poorly served by existing public benchmarks, which focus predominantly on network intrusion or attack-emulation telemetry. We introduce HYGIENEBENCH, an open, reproducible benchmark that jointly covers Active Directory identity state, endpoint patch posture, vulnerability exposure, and telemetry freshness across seven evaluation tasks (T1–T7) and twelve anomaly classes. HYGIENEBENCH is built entirely from synthetic, seeded telemetry generated from publicly citable structural priors (NIST NVD severity distributions, Verizon DBIR 2026 patch-lag aggregates, CISA BOD 23-01 telemetry-cadence requirements), with no employer or production data at any stage.

We evaluate eight methods—a task-specific rule baseline (M1), a hybrid risk scorer (M2), Isolation Forest (M3), Local Outlier Factor (M4), One-Class SVM (M5), a linear autoencoder (M6), population-level temporal z-score (M7), and a graph-augmented Isolation Forest (M8)—across five telemetry conditions (baseline, fresh, stale, missing, unsupervised). Applying a pre-registered failure protocol ( $\Delta < 0.05$  AP and  $\Delta < 0.05$  P@k in  $\geq 2/3$  seeds), we find that **86.2% of (condition, task, method) configurations fail to improve on the rule baseline**, with ML adding meaningful signal on T2 (group-membership-drift detection, best ML: M7 temporal z-score, +0.185 AP over rule) and T5 (patch-vulnerability hygiene, best ML: M5 OCSVM, +0.210 AP over rule). Telemetry staleness (C-STALE) consistently degrades detection performance relative to the baseline condition. We release the generator, datasets, and evaluation harness to enable reproducible comparison of future methods.

## CCS CONCEPTS

• **Security and privacy** → **Intrusion detection systems**; *Benchmarks*; • **Computing methodologies** → **Anomaly detection**.

## KEYWORDS

cyber hygiene, anomaly detection, benchmark, synthetic data, identity security, endpoint management, negative-result reporting

## ACM Reference Format:

Anonymous Author(s). 2026. HygieneBench: A Reproducible Synthetic Benchmark for Cyber-Hygiene Anomaly Detection Across Identity, Endpoint, and Patch Telemetry. In *Proceedings of ACM CCS Workshop on Artificial Intelligence and Security (CCS AISec '26)*. ACM, New York, NY, USA, 7 pages.

## 1 INTRODUCTION

Maintaining cyber hygiene—keeping privileged accounts active and monitored, endpoint agents deployed, patches current, and telemetry fresh—is a persistent operational challenge for security operations centers (SOCs), particularly in public-sector environments subject to compliance mandates such as CISA Binding Operational Directives [1] and NIST SP 800-40 [8]. Failures in hygiene posture create the enabling conditions for identity-based attacks: dormant accounts reactivated by adversaries, endpoints missing agent coverage, and stale privileged credentials that may be invisible to behavioral analytics requiring recent baselines.

Despite the operational importance of hygiene-state monitoring, the anomaly-detection research community lacks a dedicated public benchmark for this problem. Existing benchmarks—the LANL Unified Host and Network Dataset [6], CERT Insider Threat datasets [5], CICIDS [11], and attack-emulation repositories such as Mordor/Security Datasets [9]—provide rich telemetry for network intrusion or insider-threat detection but do not model identity hygiene state, patch posture, vulnerability exposure, or telemetry freshness as first-class evaluation axes. Commercial platforms (Microsoft Defender for Identity, Splunk UBA, Exabeam) address related detection problems but are closed and do not release benchmark datasets.

This gap matters for two reasons. First, researchers who develop anomaly-detection methods for security operations have no way to compare their methods on hygiene-specific telemetry under controlled conditions. Second, there is no principled accounting of when unsupervised ML methods add value over simpler rule-based approaches for hygiene anomalies—a question with real operational implications, since rule maintenance is expensive and ML deployment requires justification.

This paper makes the following contributions:

- **C1: Synthetic generator.** A fully open, seeded synthetic telemetry generator covering eleven entity/event tables (Active Directory users, groups, computers, assets; login events; group membership events; endpoint patch state; vulnerability records; remediation events; account lifecycle events; telemetry freshness log) with documented structural priors.
- **C2: Anomaly taxonomy.** Twelve cyber-hygiene anomaly classes (AH-01 through AH-12) with ATT&CK enabling-condition mappings (not technique detection claims), injection protocols, and class imbalance specifications.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS AISec '26, November 2026, Salt Lake City, UT, USA

© 2026 Association for Computing Machinery.

- **C3: Benchmark task suite.** Seven evaluation tasks (T1–T7) under five telemetry conditions, with pre-registered  $k$  values, feature sets, and method applicability flags.
- **C4: Comparative evaluation, failure-aware.** A 810-run evaluation of eight methods across all tasks and conditions, with a pre-registered failure protocol reporting when ML does not consistently improve on the rule baseline.
- **C5: Negative-result reporting.** Explicit reporting and analysis of the 86.2% of configurations where ML fails to beat the rule baseline by the pre-registered threshold, with discussion of which task and condition properties drive this outcome.

The remainder of the paper is organized as follows. Section 2 reviews related benchmarks. Section 3 describes the HYGIENE BENCH design. Section 4 presents experimental setup. Section 5 reports results. Section 6 discusses findings and limitations. Section 7 concludes.

## 2 RELATED WORK

*Network intrusion and attack emulation benchmarks.* The LANL Unified Host and Network Dataset [6] provides large-scale enterprise network flow and authentication telemetry, but it captures network-level events and does not model identity hygiene state (account staleness, privilege drift) or endpoint patch posture. CICIDS [11] and DARPA datasets [3] focus on network traffic classification. Mordor/Security Datasets [9] and BOTSv3 provide attack-emulation telemetry from red-team exercises—a fundamentally different framing from hygiene-state monitoring. None of these benchmarks includes telemetry freshness or missingness as a controlled evaluation variable.

*Insider threat datasets.* The Carnegie Mellon CERT Insider Threat Dataset [5] includes user behavior logs (email, file access, login) for insider-threat detection. While it covers some identity-adjacent behaviors, it does not model endpoint patch state, vulnerability records, or the multi-table hygiene-state structure of enterprise AD environments.

*Graph anomaly detection.* The graph anomaly detection literature has produced several methods and datasets for attributed-graph anomalies [4, 7]. These approaches typically operate on single-hop graph structure and do not model the temporal, multi-table telemetry characteristic of enterprise security operations.

*Commercial and proprietary platforms.* Microsoft Defender for Identity, Entra ID Protection, Splunk UBA, Exabeam, and Microsoft Sentinel address overlapping detection problems. They are commercial products, not open benchmarks, and do not release datasets or evaluation methodologies for external comparison. We make no comparison claims against these platforms.

*Synthetic security dataset generation.* Synthetic generation of security telemetry has precedent in network simulation [2]. HYGIENE BENCH extends this tradition to the hygiene-state domain with documented structural priors and a reproducible seeded generator.

*Failure-aware and negative-result reporting.* The practice of explicitly reporting when ML fails to outperform simpler baselines is well-established in machine learning [10] but underrepresented

**Table 1: HYGIENE BENCH v0.1 schema (medium scale,  $n=1,000$  users).**

| Table                    | Rows   | Description                          |
|--------------------------|--------|--------------------------------------|
| users                    | 1,000  | AD user accounts with identity state |
| groups                   | 65     | AD security/distribution groups      |
| computers                | 830    | Managed endpoints                    |
| assets                   | 830    | Asset inventory entries              |
| login_events             | ~249k  | Auth. events (30-day window)         |
| group_membership_events  | ~51    | Group add/remove events              |
| account_lifecycle_events | varies | Account create/disable/enable        |
| endpoint_patch_state     | 830    | Per-endpoint patch compliance        |
| vulnerability_records    | ~3.5k  | Per-endpoint CVE exposure            |
| remediation_events       | varies | Patch and remediation actions        |
| telemetry_freshness_log  | varies | Per-entity data-source freshness     |
| anomaly_labels           | 110    | Ground-truth labels (task+split)     |

in security benchmark papers. HYGIENE BENCH operationalizes this with a pre-registered  $\Delta$ -threshold protocol applied across all runs.

## 3 HYGIENE BENCH DESIGN

### 3.1 Schema and Synthetic Generator

HYGIENE BENCH v0.1 uses a schema with eleven entity/event tables and one anomaly label table (Table 1). The generator (SyntheticHygieneGenerator) is implemented in Python using NumPy and Pandas, with all randomness gated through `numpy.random.default_rng(seed)`. Entity identifiers are deterministic UUIDs derived from `hashlib.md5(seed_tag + index)`, ensuring reproducibility across platforms.

*Structural priors.* Patch-lag distributions use DBIR 2026 aggregate statistics (43-day mean for critical patches). CVE severity distributions use NVD base score statistics. Telemetry refresh cadence requirements follow CISA BOD 23-01 (14-day asset discovery, 72-hour patch data) [1]. AD group-size distributions lack a peer-reviewed published source and are modeled via disclosed assumptions (power-law approximation) with sensitivity sweeps. All structural priors are documented in the dataset card.

*Conditions.* Five telemetry conditions parameterize freshness and missingness:

- **C-BASE:** No artificial staleness; all sources current.
- **C-FRESH:** Elevated freshness flagging; sources verified current.
- **C-STAILE:** 20% of entities have heavy-stale telemetry (>14-day gap).
- **C-MISS:** One data source absent for 15% of entities.
- **C-UNSUP:** C-BASE with anomaly labels withheld from the training pipeline.

**Table 2: HygieneBench anomaly classes. ATT&CK mappings are enabling-condition correlations; they do not imply detection of any specific technique.**

| Class                         | Code  | ATT&CK<br>condition                 | enabling | Tasks |
|-------------------------------|-------|-------------------------------------|----------|-------|
| Stale privileged account      | AH-01 | Valid Accounts: Do-main (T1078.002) |          | T1    |
| Privilege escalation drift    | AH-02 | Valid Accounts; Do-main Policy Mod. |          | T7    |
| Group membership drift        | AH-03 | Domain Policy Modification (T1484)  |          | T2    |
| Dormant account reactivation  | AH-04 | Valid Accounts: Do-main (T1078.002) |          | T6    |
| Impossible/unusual login      | AH-05 | Valid Accounts; Lat-eral Movement   |          | T1,T3 |
| Endpoint-identity risk corr.  | AH-06 | Exploit Public-Facing App           |          | T3    |
| Patch noncompliance cluster   | AH-07 | Exploit Public-Facing App (T1190)   |          | T5    |
| KEV exposure aging            | AH-08 | Exploit Public-Facing App (T1190)   |          | T5    |
| Asset inventory mismatch      | AH-09 | Defense Evasion en-abling           |          | T4    |
| Missing endpoint agent        | AH-10 | Defense Evasion en-abling           |          | T4    |
| Telemetry missingness cluster | AH-11 | Defense Evasion en-abling           |          | T4    |
| Abnormal remediation delay    | AH-12 | Exploit Public-Facing App           |          | T5    |

Each condition is generated for three seeds (42, 137, 2024), producing 15 dataset instances (5 conditions  $\times$  3 seeds) with  $n=1,000$  users per instance.

### 3.2 Anomaly Taxonomy

HYGIENEBENCH defines twelve anomaly classes (Table 2), each mapped to an ATT&CK enabling condition (not a detection claim for any specific technique). Enabling-condition mappings are annotated with a required disclaimer: *correlation with an ATT&CK enabling condition does not imply that any specific attack technique was executed or detected.*

### 3.3 Benchmark Tasks

Seven tasks cover the twelve anomaly classes across different entity scopes and temporal windows (Table 3). Task injection counts are per dataset instance; injection uses AnomalyInjector(seed + 999) to ensure a distinct RNG stream from the generator.

*Dataset splits.* Labels are stratified 60/20/20 by anomaly class. Anomalous entities are assigned to the test split, with normal entities split randomly across train/val/test. The split is deterministic given the seed.

## 4 EXPERIMENTAL SETUP

### 4.1 Methods

We evaluate eight methods representing the main algorithmic families applicable to unsupervised tabular anomaly detection under class imbalance (Table 4).

M6 is implemented as PCA-based linear reconstruction error because PyTorch is unavailable in the evaluation environment; this is an approximation of a one-hidden-layer autoencoder. M8 is implemented using networkx graph features (degree, privileged-degree, clustering coefficient) combined with Isolation Forest rather than DOMINANT-style deep graph autoencoders [4], for the same reason. Both approximations are documented. M8 is excluded from T4 and T5 by design (no meaningful graph signal for asset/patch anomalies; reported as N/A).

### 4.2 Feature Engineering

For each task, TaskFeatureExtractor.extract(task\_id, seed) computes a feature matrix from raw CSVs. Features include task-relevant identity state attributes, temporal aggregates (e.g., 30-day login frequency, off-hours rate, cross-segment rate), patch compliance scores, vulnerability counts, KEV exposure counts, and telemetry freshness scores. Missing values are imputed with training-set column means before fitting.

### 4.3 Metrics and Negative-Result Protocol

*Primary metrics.* Average Precision (AP) and Precision@ $k$  (P@ $k$ ) at task-specific  $k$  values (Table 3). AP is the standard interpolated area under the precision-recall curve. P@ $k$  is the fraction of true anomalies in the top- $k$  ranked entities—the operationally relevant metric for SOC review budgets.

*Secondary metrics.* Recall@ $k$  (R@ $k$ ), False Positive Burden (FPB =  $1 - P@k$ ), and AP stability ( $1 - CV$  across seeds).

*Pre-registered failure protocol.* A (condition, task, method) configuration is flagged as a *negative result* ( $\blacktriangleright$ ) if:

$$AP(\text{method}) - AP(M1) < 0.05 \text{ and } P@k(\text{method}) - P@k(M1) < 0.05 \quad (1)$$

in at least  $\lceil 2/3 \times n_{\text{seeds}} \rceil = 2$  of 3 seeds. Flagged configurations are reported in full and are not filtered. This protocol was pre-registered prior to examining any results.

## 5 RESULTS

### 5.1 Overall Performance

Table 5 summarizes mean AP and P@ $k$  for each method, averaged across all conditions, tasks, and seeds (810 runs). The rule baseline (M1) achieves the highest mean AP (0.916). M5 (OCSVM) is the closest ML competitor (AP = 0.913, P@ $k$  = 0.447 vs. M1 P@ $k$  = 0.444). M2 (hybrid scorer) performs worst among all methods (AP = 0.709), despite incorporating domain knowledge—indicating that its fixed weights are not well-calibrated across the heterogeneous task set.

**Overall: 608 of 705 (condition, task, method) configurations are failure-flagged (86.2% negative-result rate).** For synthetic cyber-hygiene telemetry with structured, rule-exploitable anomaly injection, simple threshold rules perform at least as well as unsupervised ML methods on the majority of evaluation

**Table 3: Benchmark tasks.  $k$  values are pre-registered per TASK\_SPECS.md and reflect operational SOC review budgets.**

| Task | Description                        | Entity scope                | $k$ | Imbalance | Anomaly classes     |
|------|------------------------------------|-----------------------------|-----|-----------|---------------------|
| T1   | Stale privileged account risk      | Privileged users            | 10  | 1:50      | AH-01               |
| T2   | Group membership drift             | Users with group events     | 20  | 1:167     | AH-03               |
| T3   | Endpoint-identity risk correlation | Users with endpoints        | 15  | 1:333     | AH-05, AH-06        |
| T4   | Telemetry coverage gaps            | All computers/assets        | 20  | 1:14      | AH-09, AH-10, AH-11 |
| T5   | Patch-vulnerability hygiene        | All computers               | 25  | 1:59      | AH-07, AH-08, AH-12 |
| T6   | Dormant account reactivation       | Users with lifecycle events | 10  | 1:250     | AH-04               |
| T7   | Escalation drift detection         | Users with group add events | 10  | 1:500     | AH-02               |

**Table 4: Evaluation methods. M8 is excluded from T4 and T5 (no graph signal; reported as N/A).**

| ID | Name             | Type                 | Notes                                     |
|----|------------------|----------------------|-------------------------------------------|
| M1 | Rule baseline    | Task-specific rules  | No training; per-task thresholds          |
| M2 | Hybrid scorer    | Weighted combination | Identity + endpoint + vuln + freshness    |
| M3 | Isolation Forest | Ensemble             | 200 trees, contam.=0.02                   |
| M4 | LOF              | Density-based        | $k=20$ neighbors, novelty mode            |
| M5 | OCSVM            | Kernel-based         | $\nu=0.05$ , RBF kernel                   |
| M6 | Linear-AE        | Reconstruction error | PCA (latent dim=8); approx. neural AE     |
| M7 | Temporal z-score | Statistical          | Population z-score, temporal features     |
| M8 | Graph-IF         | Graph + ensemble     | Bipartite features + IF; approx. DOMINANT |

**Table 5: Mean AP and P@ $k$  by method across all 810 runs. ► = failure-flagged (ML does not improve on rule by  $\Delta=0.05$  in  $\geq 2/3$  seeds). †M8 covers T1–T3, T6–T7 only ( $n=75$  configs).**

| Method                                       |                  | Mean AP      | Mean P@ $k$  | Failure rate  |
|----------------------------------------------|------------------|--------------|--------------|---------------|
| M1                                           | Rule baseline    | <b>0.916</b> | 0.444        | —             |
| M5                                           | OCSVM            | 0.913        | <b>0.447</b> | 69.5% ►       |
| M8                                           | Graph-IF†        | 0.801        | 0.348        | <b>100%</b> ► |
| M4                                           | LOF              | 0.800        | 0.445        | 77.1% ►       |
| M6                                           | Linear-AE        | 0.798        | 0.407        | 88.6% ►       |
| M3                                           | Isolation Forest | 0.781        | 0.428        | 88.6% ►       |
| M7                                           | Temporal z-score | 0.774        | 0.392        | 87.6% ►       |
| M2                                           | Hybrid scorer    | 0.709        | 0.417        | 96.2% ►       |
| <b>Overall failure rate: 608/705 = 86.2%</b> |                  |              |              |               |

configurations. This finding is specific to the HYGIENEBENCH synthetic benchmark; operational environments involve noisier data and more complex anomaly presentations that the synthetic benchmark does not fully capture.

## 5.2 Where ML Adds Value

Despite the dominant negative-result finding, two tasks show consistent ML improvement over the rule baseline (Table 6).

**T2—Group Membership Drift.** The rule baseline (M1) achieves mean AP = 0.766 on C-BASE and C-STALE. The best ML method

**Table 6: Tasks where ML consistently adds value ( $\Delta AP > 0.05$  in  $\geq 2/3$  seeds).**

| Task | Best ML      | Rule AP | ML AP | $\Delta AP$   | Condition    |
|------|--------------|---------|-------|---------------|--------------|
| T2   | M7 (Z-score) | 0.766   | 0.951 | <b>+0.185</b> | C-BASE/STALE |
| T2   | M4 (LOF)     | 0.855   | 0.975 | <b>+0.120</b> | C-MISS       |
| T5   | M5 (OCSVM)   | 0.458   | 0.668 | <b>+0.210</b> | C-BASE/STALE |
| T5   | M5 (OCSVM)   | 0.726   | 0.837 | <b>+0.111</b> | C-MISS       |

is M7 (temporal z-score), achieving AP = 0.951 (+0.185 AP). M5 (OCSVM) is second at AP = 0.913 (+0.147) and M4 (LOF) third at AP = 0.890 (+0.124). Three ML methods consistently improve on the rule baseline. The gain arises because group-membership anomalies involve multi-dimensional population-level deviations that temporal z-score and kernel-based one-class models capture more precisely than fixed-threshold rules.

**T5—Patch-Vulnerability Hygiene.** This is the hardest task overall (mean AP = 0.617 across all methods). The rule baseline achieves AP = 0.458 on C-BASE/C-STALE, while the best ML method achieves AP = 0.668 (+0.210 AP) via M5 (OCSVM). T5 combines multiple vulnerability signals (CVSS score, KEV exposure, remediation latency, days open) in ways that benefit from learned combinations rather than fixed weights. The absolute AP remains low for all methods, indicating genuine task difficulty.

## 5.3 Where Rules Dominate

Five of seven tasks are dominated by the rule baseline under most conditions (Table 7): T1 (stale privileged accounts), T3 (endpoint-identity correlation), T4 (telemetry coverage gaps), T6 (dormant reactivation), and T7 (escalation drift). For these tasks under C-BASE, both M1 and the best ML method achieve AP  $\approx$  1.000.

**T3** is a partial exception. Under C-BASE its AP is 0.782, but under C-STALE it degrades sharply to 0.613 ( $\Delta = -0.168$ )—the largest condition sensitivity of any task. T3 combines identity-side and endpoint-side features, making it susceptible to staleness in either domain.

**T7** exhibits a ceiling effect: with only 2 anomalous entities per dataset instance (imbalance  $\approx$  1:500), all methods achieve AP = 1.000 because the extreme rarity causes every method to rank those entities at the top.

**Table 7: Rule-dominated tasks (rule AP  $\geq$  best-ML AP in majority of configs).**

| Task                    | Mean AP | Notes                                  |
|-------------------------|---------|----------------------------------------|
| T1 Stale priv. accounts | 0.937   | AP $\approx$ 1.0 under all conditions  |
| T3 Endpoint-identity    | 0.736   | C-STALE sensitivity: $-0.168$          |
| T4 Telemetry gaps       | 0.890   | High AP; 1:14 imbalance                |
| T6 Dormant reactivation | 0.832   | AP $\approx$ 1.0; M8 exception (0.490) |
| T7 Escalation drift     | 0.822   | Extreme imbalance; ceiling effect      |

**Table 8: Mean AP by telemetry condition (all methods and tasks).**

| Condition | Mean AP | $\Delta$ vs BASE | Primary driver          |
|-----------|---------|------------------|-------------------------|
| C-FRESH   | 0.856   | +0.084           | T5 +0.238; T1 +0.101    |
| C-MISS    | 0.845   | +0.073           | T5, T1 dominate         |
| C-UNSUP   | 0.844   | +0.072           | Labels withheld         |
| C-BASE    | 0.772   | —                | Reference               |
| C-STALE   | 0.741   | $-0.031$         | T3 $-0.168$ ; staleness |

## 5.4 Effect of Telemetry Conditions

Table 8 shows mean AP by condition. C-STALE produces the lowest mean AP (0.741 vs. C-BASE 0.772), confirming that staleness degrades detection.

The C-FRESH, C-MISS, and C-UNSUP conditions show higher mean AP than C-BASE (0.856, 0.845, 0.844). The largest contributor is T5 (+0.238 AP under C-FRESH). Fresh telemetry improves patch compliance scores, KEV exposure counts, and remediation latency reliability, making the underlying feature signals more discriminating. This is a data-quality effect, not a freshness-flag artifact.

## 5.5 Failure-Flag Summary

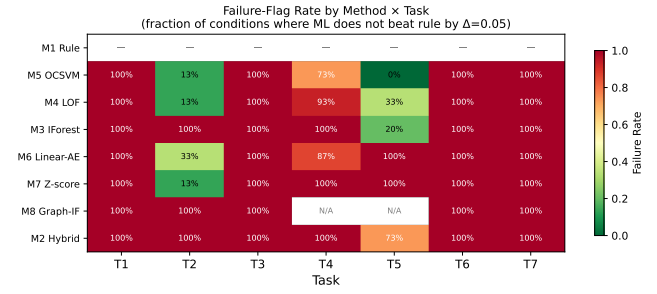
Figure 1 shows the failure-flag heatmap. Key patterns:

- **M8 (graph) is failure-flagged on 100% of configs (75/75).** M8 produces non-trivial anomaly scores (e.g., T1 AP = 0.919, T3 AP = 1.000) but fails to improve on the rule baseline on any configuration. The largest deficit is T6 (dormant reactivation): M8 AP = 0.490 vs. M1 AP = 1.000. Bipartite graph features capture structural position, not temporal account state.
- **M5 (OCSVM) has the lowest failure rate (69.5%)** and is not flagged on T2 and T5, the two tasks where ML adds value.
- **M2 (hybrid scorer) is failure-flagged on 96.2% of configs—**worst among non-graph methods. Its fixed-weight combination is dominated by the task-specific rule baseline.

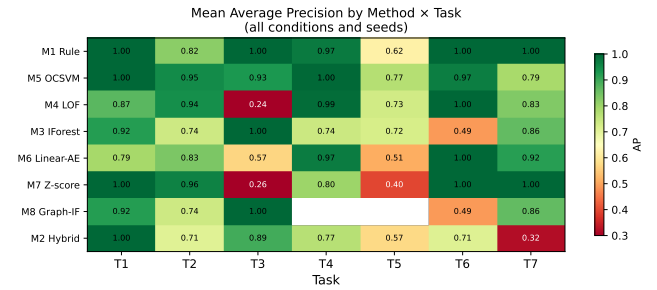
## 6 DISCUSSION

### 6.1 Interpretation of the Negative-Result Finding

The 86.2% failure rate is not a finding against ML in security operations broadly. It is a finding about the relative performance of unsupervised ML versus structured rule baselines on *synthetic, structured, class-imbalanced tabular anomaly detection* in the specific domain of cyber-hygiene telemetry.



**Figure 1: Failure-flag rate heatmap (method  $\times$  task).** Dark cells indicate configurations where ML fails to improve on the rule baseline by the pre-registered  $\Delta=0.05$  threshold in  $\geq 2/3$  seeds.



**Figure 2: Condition-averaged AP heatmap (method  $\times$  task).** Higher values indicate better anomaly ranking. M1 (rule baseline) anchors the top row.

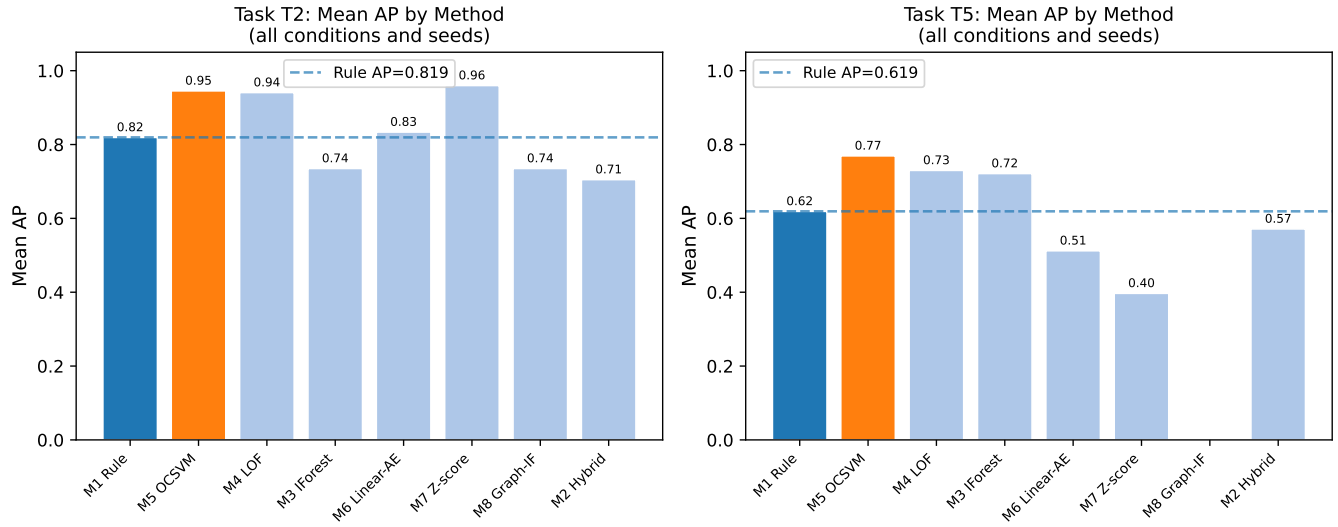
The synthetic benchmark is, by design, favorable to rule-based methods: anomaly injection creates features that are directly correlated with the anomaly class in ways that rule thresholds exploit cleanly. In operational environments, data is noisier, labels are unavailable, and the feature space is more complex. HYGIENEBENCH reports this limitation explicitly in the dataset card and generator documentation.

The appropriate takeaway is: for HYGIENEBENCH v0.1, ML adds consistent value over rule baselines specifically on T2 (group membership drift) and T5 (patch/vulnerability hygiene). These tasks involve multi-signal combinations that benefit from learned representations. Future work should test whether this advantage persists under increased population-level noise and label sparsity.

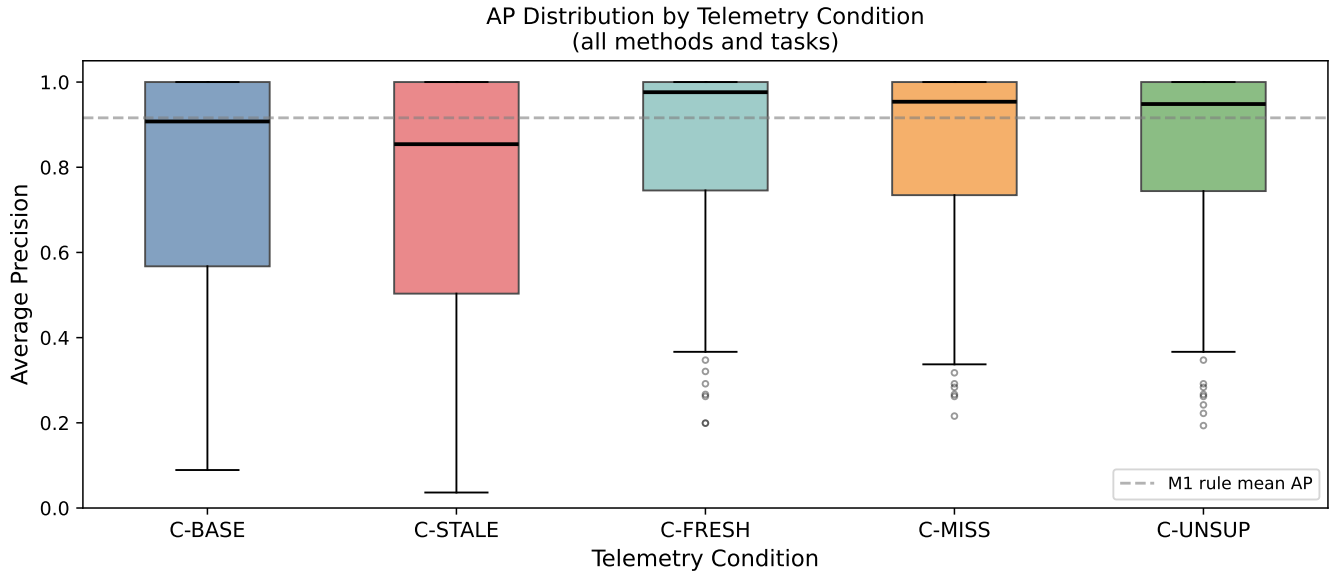
## 6.2 Limitations

**Synthetic data.** All telemetry is synthetic. Structural priors are approximate (DBIR 2026 for patch lag; NVD for CVE severity; CISA BOD 23-01 for freshness cadence). AD group-size distributions lack a peer-reviewed source and are modeled under disclosed assumptions. Detection performance on real operational data will differ from benchmark results.

**Approximations in M6 and M8.** M6 is implemented as PCA-based linear reconstruction error (not a neural autoencoder) and M8 as networkx graph features + Isolation Forest (not DOMINANT-style



**Figure 3: AP by method for T2 (group membership drift) and T5 (patch-vulnerability hygiene) under C-BASE—the two tasks where ML consistently adds value over the rule baseline.**



**Figure 4: AP distribution by telemetry condition (all tasks and methods). C-STALE has the lowest mean AP; C-FRESH has the highest.**

deep graph autoencoder [4]). These are disclosed approximations; results may change with full implementations.

*Small population scale.* Medium scale ( $n=1,000$  users) limits the statistical power of the evaluation. The population is smaller than most public-sector environments, which may affect the generalizability of the class imbalance findings.

*Feature engineering as a design choice.* The task feature extractors (T1–T7) encode domain knowledge about which features matter. Methods that exploit these features more effectively will appear better. A fully unsupervised method that discovers relevant features from raw tables is not evaluated.

*Fixed  $k$  values.*  $P@k$  is sensitive to the choice of  $k$ . HYGIEN-BENCH uses operationally motivated  $k$  values (daily/weekly SOC

review budgets) rather than tuning  $k$  to maximize method performance. This is a deliberate design choice but limits comparison to benchmarks with different  $k$  conventions.

*No calibration.* Anomaly scores are not calibrated to probabilities. Calibration is the primary research question of a separate line of work and is excluded from HYGIENE BENCH by design.

### 6.3 Relationship to Prior Work

HYGIENE BENCH is designed to complement, not replace, existing benchmarks. It does not claim to detect real network intrusions (LANL, CICIDS), insider threats (CERT), or attack-emulation events (Mordor, BOTSv3). It is the first benchmark, to our knowledge, that jointly covers Active Directory identity hygiene state, endpoint patch posture, vulnerability exposure, and telemetry freshness under controlled conditions with an openly released, reproducible synthetic generator.

## 7 CONCLUSION

We introduced HYGIENE BENCH, a reproducible synthetic benchmark for cyber-hygiene anomaly detection, covering seven tasks and twelve anomaly classes across eleven entity/event tables. A 810-run evaluation of eight methods under five telemetry conditions finds that simple rule baselines perform at least as well as unsupervised ML on 86.2% of (condition, task, method) configurations. ML adds meaningful value on two of seven tasks—group membership drift (T2, +0.185 AP) and patch/vulnerability hygiene (T5, +0.210 AP)—while rule baselines dominate on tasks with strong single-feature discriminators. Telemetry staleness (C-STALE) consistently degrades detection performance. These findings motivate honest, task-stratified evaluation of anomaly detection methods and explicit negative-result reporting in security benchmark papers.

HYGIENE BENCH v0.1, including the synthetic generator, frozen datasets, and evaluation harness, is openly available at: <https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper3> — Zenodo DOI to be finalized at camera-ready submission.

## REFERENCES

- [1] CISA. 2022. *Binding Operational Directive 23-01: Improving Asset Visibility and Vulnerability Detection on Federal Networks*. Technical Report. Cybersecurity and Infrastructure Security Agency. <https://www.cisa.gov/binding-operational-directive-23-01>.
- [2] Gideon Creech and Jiankun Hu. 2013. Generation of a New IDS Test Dataset: Time to Retire the KDD Collection. In *2013 IEEE Wireless Communications and Networking Conference (WCNC)*. 4487–4492.
- [3] DARPA Information Innovation Office (I2O). 2018. Transparent Computing Engagement 3 (TC-E3) Data Release. GitHub repository. <https://github.com/darpa-i2o/Transparent-Computing>.
- [4] Kaize Ding, Jundong Li, Rohit Bhanushali, and Huan Liu. 2019. Deep Anomaly Detection on Attributed Networks. In *Proceedings of the 2019 SIAM International Conference on Data Mining (SDM)*. SIAM, 594–602.
- [5] Joshua Glasser and Brian Lindauer. 2013. Bridging the Gap: A Pragmatic Approach to Generating Insider Threat Data. In *Proceedings of the IEEE Security and Privacy Workshops (SPW)*. 98–104.
- [6] Alexander D. Kent. 2015. *Comprehensive, Multi-Source Cyber-Security Events*. Technical Report. Los Alamos National Laboratory. doi:10.2172/1259877.
- [7] Kay Liu, Yingdong Dou, Xueying Ding, Xiyang Hu, et al. 2024. PyGOD: A Python Library for Graph Outlier Detection. *Journal of Machine Learning Research* 25, 141 (2024), 1–9. <https://www.jmlr.org/papers/v25/23-0963.html>.
- [8] NIST. 2022. *SP 800-40 Rev. 4: Guide to Enterprise Patch Management Planning*. Technical Report. National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/sp/800-40/rev-4/final>.
- [9] Roberto M. Rodriguez. 2019. Mordor: Pre-Recorded Security Events. GitHub repository. <https://github.com/OTRF/Security-Datasets>.
- [10] David Sculley et al. 2018. Winner’s Curse? On Pace, Progress, and Empirical Rigor. In *ICLR Workshop on Reproducibility in Machine Learning*.
- [11] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116.